

Earthquake Co-occurrence Analysis: Strong Scalability Study

3 febbraio 2026

1 Introduzione

Questo progetto implementa un'applicazione in Scala per l'analisi di co-occorrenza di eventi sismici utilizzando Spark su Google Cloud Platform (Dataproc). L'obiettivo è identificare coppie di località geografiche che sperimentano terremoti nello stesso giorno e valutare la scalabilità dell'applicazione su cluster di diverse dimensioni. Sono state testate due configurazioni di macchine virtuali (n2-standard-4 e n1-standard-2) con benchmarking automatizzato tramite script PowerShell.

2 Setup Infrastruttura Google Cloud

Prima di eseguire il benchmarking, è stato necessario configurare l'infrastruttura su Google Cloud Platform tramite lo script PowerShell `setup_gcloud.ps1`, che automatizza i seguenti compiti:

1. **Configurazione Progetto GCP:** imposta il progetto `scalableproject-482714`
2. **Creazione Bucket Cloud Storage:** crea `terremoti-bucket-giorgiapirelli` con cartelle dedicate (`jars/`, `output/`)
3. **Build Automatico con SBT:** compila con `sbt clean` e `sbt package`
4. **Upload JAR in Cloud Storage:** carica il JAR compilato
5. **Verifica Dataset:** Controlla la presenza del dataset

Lo script deve essere eseguito una sola volta prima del benchmarking.

3 Workflow del Sistema

Il workflow viene sviluppato:

1. **Creazione Cluster:** script PowerShell crea cluster Dataproc con nodo master + N worker
2. **Esecuzione Job:** lancia l'applicazione via `gcloud dataproc jobs submit spark`
3. **Misurazione Performance:** registra tempi di esecuzione in CSV
4. **Pulizia:** elimina il cluster automaticamente dopo il job

4 Approccio Implementativo

L'applicazione implementa quattro fasi di elaborazione:

4.1 Fase 1: Parsing e Trasformazione

Estrazione di (data, latitudine, longitudine) da CSV, filtro header, arrotondamento coordinate alla prima cifra decimale.

4.2 Fase 2: Partizionamento

Utilizzo di HashPartitioner per distribuzione uniforme per data:

```
1 val partitioned = data.partitionBy(new HashPartitioner(numPartitions))
2 val grouped = partitioned.groupByKey()
```

Garantisce: distribuzione uniforme.

4.3 Fase 3: Generazione Coppie e Ricerca del Massimo

Per ogni data, vengono generate tutte le possibili coppie di locazioni co-occorrenti. Le fasi specifiche sono:

- **Generazione:** tramite `flatMap`, vengono create tutte le combinazioni non ordinate di locazioni per ogni data
- **Deduplicazione:** le locazioni vengono trasformate in Set per eliminare automaticamente i duplicati
- **Raggruppamento:** con `groupByKey()`, tutte le date associate a ogni coppia vengono raccolte insieme
- **Ricerca del Massimo:** mediante l'operazione `reduce()`, viene identificata la coppia che co-occorrerà nel maggior numero di date distinte

4.4 Fase 4: Output

Durante l'esecuzione del job, l'applicazione genera un output che identifica la coppia di locazioni con il massimo numero di co-occorrenze. La coppia vincente viene stampata nei log del job Spark, mentre la lista completa delle date di co-occorrenza viene salvata in un file di output.

4.5 Stampa della Coppia Vincente nei Log

La coppia vincente e il numero totale di co-occorrenze vengono stampati nei log del job Spark durante l'esecuzione:



Figura 1: La coppia di locazioni $(-122.8, 38.8)$ e $(-122.7, 38.8)$ viene stampata nei log insieme al numero totale di co-occorrenze (10.014). L'output è visibile nella scheda "Riepilogo" del job.

5 Configurazioni di Test

Durante i test iniziali con `n2-standard-4`, si è riscontrato un errore di quota GCP nella creazione del cluster con 4 nodi worker, causando il fallimento della creazione del cluster con tale configurazione.

```
ERROR: (gcloud.dataproc.clusters.create) INVALID_ARGUMENT: Multiple validation errors:
- Insufficient 'CPUS_ALL_REGIONS' quota. Requested 20.0, available 16.0. Your resource request exceeds your available q
uota. See https://cloud.google.com/compute/resource-usage. Use https://cloud.google.com/docs/quotas/view-manage#requesti
ng_higher_quota to request additional quota.
- This request exceeds CPU quota. Some things to try: request fewer workers (a minimum of 2 is required), use smaller m
aster and/or worker machine types (such as n1-standard-2).
```

Figura 2: errore di quota GCP, 20 CPU richiesti vs 16 CPU disponibili nel progetto

A causa di questo vincolo di quota GCP, è stata adottata una strategia di test alternativa e più completa:

1. **Test con `n2-standard-4`:** limitati a 2-3 worker con dataset completo per rispettare il vincolo di quota
2. **Test con `n1-standard-2`:** utilizzata come configurazione alternativa con 2 vCPU, 7.5 GB RAM, 100 GB disk per nodo. Questa scelta ha permesso test più estesi:
 - **Dataset piccolo:** 2-4 worker con 4/8/12 partizioni
 - **Dataset completo:** 2-4 worker con 16/24 partizioni

Sebbene il progetto richiedesse l'utilizzo del solo dataset completo, si è scelto di includere anche i risultati per il dataset piccolo con `n1-standard-2` per fornire un'analisi comparativa più completa.

5.1 Motivazione dei test con più partizioni

I vari test e prove sono stati eseguiti aumentando progressivamente il numero di partizioni perché **si riduceva il tempo di esecuzione**. Invece, se si manteneva troppo basso il numero di partizioni, la riduzione dei tempi **non era significativa**. Per questo motivo, i test sono stati eseguiti partendo da **poche partizioni (4)** fino a **molte partizioni (24)** per esplorare sistematicamente l'intero range.

Esempi dal range completo testato:

- **n2-standard-4 con 2 worker** (range 8-24):
 - 8 partizioni: 923.05s
 - 16 partizioni: 641.63s (-30.4%, miglioramento)
 - 24 partizioni: 397.42s (-38.1%, continua a migliorare)
- **n1-standard-2 dataset completo con 2 worker** (range 16-24):
 - 16 partizioni: 2412.54
 - 24 partizioni: 888.59s (-63.1%, miglioramento significativo)
- **n1-standard-2 dataset piccolo con 2 worker** (range 4-12):
 - 4 partizioni: 1904.07s
 - 8 partizioni: 1172.29s (-38.4%, miglioramento)
 - 12 partizioni: 523.30s (-55.4%, continua a migliorare)

Questo approccio sistematico ha permesso di scoprire che aumentare progressivamente le partizioni produceva consistenti riduzioni dei tempi di esecuzione, giustificando l'esecuzione di test con il range completo (da 4 a 24 partizioni).

6 Metodologia di calcolo: strong scalability su GCP Dataproc

6.1 Formula standard vs Formula GCP Dataproc

La metrica di efficienza di scalabilità è definita come:

$$E(n) = \frac{S(n)}{n} = \frac{T(1)}{n \times T(n)} \quad (1)$$

Teoricamente, $E(n) \leq 100\%$ rappresenta la scalabilità lineare perfetta. Tuttavia, il valore $T(1)$ non è disponibile sperimentalmente.

6.2 Soluzione: Formula Alternativa per GCP Dataproc

Quindi si utilizza $T(2)$ (tempo con 2 worker) come baseline, poiché è il minimo misurabile. La formula diventa:

$$E(n) = \frac{S(n)}{n} \times 2 = \frac{T(2)}{n \times T(n)} \times 2 = \frac{2}{n} \times S(n) \quad (2)$$

dove $S(n) = T(2)/T(n)$. **Esempio Numerico:** con 3 worker: $E(3) = (2/3) \times 1.62 \times 100 = 108.0\%$

7 Analisi di Scalabilità

7.1 Risultati n1-standard-2 (dataset piccolo)

Tabella 1: Benchmark n1-standard-2 - dataset piccolo

W	P	Durata (s)	Speedup	Eff
2	4	1904.07	1.00	100.0%
2	8	1172.29	1.00	100.0%
2	12	523.30	1.00	100.0%
3	4	1429.39	1.33	88.7%
3	8	753.40	1.56	104.0%
3	12	431.61	1.21	80.7%
4	4	995.01	1.91	95.5%
4	8	708.06	1.65	82.5%
4	12	349.71	1.50	75.0%

7.2 Risultati n1-standard-2 (dataset completo)

I test con n1-standard-2 sono stati estesi al dataset completo.

Tabella 2: Benchmark n1-standard-2 - dataset completo

W	P	Durata (s)	Speedup	Eff
2	16	2412.54	1.00	100.0%
2	24	888.59	1.00	100.0%
3	16	1489.71	1.62	108.0%
3	24	640.86	1.39	92.7%
4	16	1109.71	2.17	108.5%
4	24	492.26	1.81	90.5%

7.2.1 Analisi dettagliata per numero di partizioni - n1-standard-2 dataset completo

Tabella 3: Scalabilità con 16 partizioni (n1-standard-2 - dataset completo)

Workers	Durata (s)	Speedup	Efficienza
2	2412.54	1.00	100.0%
3	1489.71	1.62	108.0%
4	1109.71	2.17	108.5%

Tabella 4: Scalabilità con 24 partizioni (n1-standard-2 - dataset completo)

Workers	Durata (s)	Speedup	Efficienza
2	888.59	1.00	100.0%
3	640.86	1.39	92.7%
4	492.26	1.81	90.5%

7.3 Risultati n2-standard-4 (dataset completo)

Questo set di test con n2-standard-4 e dataset completo mostra i risultati con configurazioni di partizioni da 8 a 24.

Tabella 5: Benchmark complessivo n2-standard-4 - Tutte le partizioni

Workers	Partizioni	Durata (s)	Speedup	Efficienza
2	8	911.99	1.00	100.0%
2	16	537.13	1.00	100.0%
2	24	389.19	1.00	100.0%
3	8	772.00	1.18	78.7%
3	16	450.10	1.19	79.3%
3	24	284.99	1.37	91.3%

Osservazioni n2-standard-4 completo: configurazione ottimale con 24 partizioni + 3 worker (speedup 1.37x, efficienza 91.3%)

7.3.1 Analisi dettagliata per partizioni - n2-standard-4 dataset completo

Tabella 6: Scalabilità con 8 partizioni (n2-standard-4 - Dataset Completo)

Workers	Durata (s)	Speedup	Efficienza
2	911.99	1.00	100.0%
3	772.00	1.18	78.7%

Tabella 7: Scalabilità con 16 partizioni (n2-standard-4 - Dataset Completo)

Workers	Durata (s)	Speedup	Efficienza
2	537.13	1.00	100.0%
3	450.10	1.19	79.3%

Tabella 8: Scalabilità con 24 partizioni (n2-standard-4 - Dataset Completo)

Workers	Durata (s)	Speedup	Efficienza
2	389.19	1.00	100.0%
3	284.99	1.37	91.3%

7.3.2 Analisi Dettagliata per Numero di Partizioni - n2-standard-4

Tabella 9: Scalabilità con 8 partizioni (n2-standard-4)

Workers	Durata (s)	Speedup	Efficienza
2	923.05	1.00	100.0%
3	883.41	1.04	69.3%

Tabella 10: Scalabilità con 16 partizioni (n2-standard-4)

Workers	Durata (s)	Speedup	Efficienza
2	641.63	1.00	100.0%
3	526.52	1.22	81.3%

Tabella 11: Scalabilità con 24 partizioni (n2-standard-4)

Workers	Durata (s)	Speedup	Efficienza
2	397.42	1.00	100.0%
3	288.43	1.38	92.0%

7.4 Seconda prova con n2-standard-4

Questo secondo set di test con n2-standard-4 e dataset completo mostra i risultati con configurazioni di partizioni da 4 a 24.

Tabella 12: Benchmark completo n2-standard-4 - Tutte le partizioni

Workers	Durata (s)	Speedup	Efficienza
2	1034.07	1.00	100.0%
2	920.30	1.00	100.0%
2	583.71	1.00	100.0%
2	400.04	1.00	100.0%
3	862.90	1.20	80.0%
3	773.76	1.19	79.3%
3	380.04	1.54	102.7%
3	275.39	1.45	96.7%

Osservazioni n2-standard-4 completo: configurazione ottimale con 16 partizioni + 3 worker (speedup 1.54x, efficienza 102.7%)

7.4.1 Analisi Dettagliata per Partizioni - n2-standard-4 completo

Tabella 13: Scalabilità con 4 partizioni (n2-standard-4)

Workers	Durata (s)	Speedup	Efficienza
2	1034.07	1.00	100.0%
3	862.90	1.20	80.0%

Tabella 14: Scalabilità con 8 partizioni (n2-standard-4)

Workers	Durata (s)	Speedup	Efficienza
2	920.30	1.00	100.0%
3	773.76	1.19	79.3%

Tabella 15: Scalabilità con 16 partizioni (n2-standard-4)

Workers	Durata (s)	Speedup	Efficienza
2	583.71	1.00	100.0%
3	380.04	1.54	102.7%

Tabella 16: Scalabilità con 24 partizioni (n2-standard-4)

Workers	Durata (s)	Speedup	Efficienza
2	400.04	1.00	100.0%
3	275.39	1.45	96.7%

7.4.2 Analisi Comparativa: Tutte le Configurazioni

La Tabella 17 confronta i risultati delle configurazioni ottimali tra dataset piccolo n1-standard-2, dataset completo n1-standard-2, e dataset completo n2-standard-4.

Tabella 17: Confronto configurazioni ottimali

Machine	Dataset	Config Opt	Speedup	Efficienza
n1-standard-2	Piccolo	8p + 3w	1.56x	104.0%
n1-standard-2	Completo	16p + 4w	2.17x	108.5%
n2-standard-4	Completo	24p + 3w	1.38x	92.0%

8 Codice Sorgente:

<https://github.com/giorgiapirelli/ScalableProjectGiorgiaPirelli.git>